

Database & SQL Fundamentals

Comprehensive Study Guide, Data Visualization & Key Cheat Sheets

🎯 Module Learning Objectives

- Explain what a database is and how software systems utilize it for persistent data management.
- Understand relational structure, table organization, and data relationship modeling.
- Differentiate between types of databases (Relational vs. NoSQL) and understand key attributes.
- Apply foundational SQL syntax to create, read, update, and manage database objects and records.

1. 📊 Data Charts & Visualizations

Charts transform raw database records into visual formats, making complex patterns easily understandable for decision-making.

CHART TYPE	PRIMARY USAGE & FUNCTION	KEY TAKEAWAY
Bar Chart 📊	Compares discrete categorical values using rectangular bars.	Taller bar = Larger value
Bubble Chart 🫧	Compares values along dimensions using bubble sizes.	Bigger bubble = Higher volume/size
Line Chart 📈	Tracks continuous data trends and changes over time.	Connected points = Trend over time
Pie Chart 🥧	Displays proportional relationships of parts to a whole.	Total slices = 100% of whole
Area Chart	Combines line and bar charts to display cumulative totals over time.	Filled area = Visual quantity
Gantt Chart	Schedules project timelines, task duration, and dependencies.	Horizontal bars = Project tasks
Heat Map	Visualizes data density or intensity using color scales.	Darker/Warm color = Higher intensity
Scatter Plot	Explores correlations and relationships between numeric variables.	Clustered points = Correlation

🧠 Quick Memory Formula for Charts:

Bar = Compare | **Bubble = Size** | **Line = Trend** | **Pie = Percentage/Whole** | **Scatter = Relationship**

2. 📅 Database Evolution Timeline

Database architectures evolved rapidly to handle increasing scale, structural complexity, and real-time operational demands.

ERA	DATABASE MODELS DEVELOPED	KEY FEATURES
1960s	Computerized Databases	Automated basic data storage, replacing manual paper indexing.
1970s – 1990s	Flat-file, Hierarchical, Network	Introduced structured parent-child trees and complex graph pointers.
1980s – Present	Relational (RDBMS)	Invented by E. F. Codd. Utilizes tables, schema integrity, and SQL.
1990s – Present	Object-oriented, Web-enabled, NoSQL	Supports non-relational, highly scalable, unstructured JSON data.

3. 📁 Core Database Models

Flat-File Database

- Stores data in a single file/table (e.g., CSV, plain text, TSV).
- Fields are separated by delimiters like commas, tabs, or spaces.
- **Limitation:** Cannot connect multiple tables or establish foreign relationships.
- **Remember: Flat = One File**

Hierarchical Database

- Organizes data in an upside-down **tree-like structure**.
- Uses **Parent** → **Child** strict relationships (primarily 1-to-Many).
- A child node can only have a single parent node.
- **Remember: Hierarchical = Tree Structure**

Network Database

- Extends the hierarchical model into a flexible graph structure.
- Supports **Many-to-Many (M:N)** relationships.
- A child node can have multiple parent nodes.
- **Remember: Network = Graph with Many Connections**

4. 🔗 Relational Database Management Systems (RDBMS)

A relational database organizes data into structured **tables** consisting of rows and columns, linked together by relational keys.

- **Rows (Records/Tuples):** Represent a single, complete data entry.
- **Columns (Fields/Attributes):** Define specific attributes and assigned data types.
- **Primary Key (PK):** Uniquely identifies each record in a table.
- **Foreign Key (FK):** Establishes a logical link between tables.

Example Relational Structure:

STUDENT Table

ID (PK)	Name
1	John
2	Sara

COURSE Table

CourseID (PK)	StudentID (FK)	Course
101	1	Python
102	2	SQL

 **Relational Summary: Relational = Tables + Strict Schema + Keys + SQL Syntax**

5. NoSQL & Database Comparison

NoSQL (Not Only SQL) databases accommodate unstructured data, dynamic schemas, horizontal scaling, and massive throughput.

4 Primary NoSQL Data Store Models

- **Document Store:** Encapsulates data in JSON or BSON documents (e.g., MongoDB).
- **Key-Value Store:** Uses associative arrays with unique key lookups (e.g., Redis).
- **Wide-Column Store:** Organizes data in dynamic column families (e.g., Cassandra).
- **Graph Store:** Connects nodes, edges, and properties for complex networks (e.g., Neo4j).

SQL vs. NoSQL Feature Matrix

FEATURE	SQL / RELATIONAL	NOSQL / NON-RELATIONAL
Structure	Structured Tables (Rows & Columns)	Flexible (Documents, Key-Value, Graphs)
Schema	Fixed / Rigid Schema	Dynamic / Flexible Schema
Relationships	Enforced via Foreign Keys	Fewer/Denormalized embedded structures
Scalability	Vertical (Scale Up)	Horizontal (Scale Out / Distributed)
Primary Example	MySQL, PostgreSQL, Oracle	MongoDB, Cassandra, Redis

6. SQL Commands & CRUD Operations

SQL manages RDBMS through standardized sub-languages corresponding to persistent operations.

SUB-LANGUAGE	FULL NAME	PURPOSE & SCOPE	COMMANDS
DDL	Data Definition Language	Defines & alters database structure	CREATE , ALTER , DROP
DML	Data Manipulation Language	Manages data inside tables	INSERT , UPDATE , DELETE
DQL	Data Query Language	Retrieves/Reads stored data	SELECT

SUB-LANGUAGE	FULL NAME	PURPOSE & SCOPE	COMMANDS
DCL	Data Control Language	Manages permissions & user access	GRANT , REVOKE

Mapping CRUD Operations to SQL Syntax

- **C — Create:** INSERT INTO TableName ...
- **R — Read:** SELECT * FROM TableName;
- **U — Update:** UPDATE TableName SET column = value ...
- **D — Delete:** DELETE FROM TableName WHERE ...

7. 🗝️ Relational Table Keys Reference

Keys ensure entity integrity, uniquely identify rows, and establish logical connections between entities.

KEY CONCEPT	DEFINITION & RULES	PRACTICAL EXAMPLE
Key Attribute	Any column used to uniquely identify records.	staff_id
Candidate Key	Any column or combination that <i>could</i> serve as a primary key.	staff_id , phone_number
Primary Key (PK)	The selected candidate key chosen to uniquely identify each row. Must be unique and non-null.	staff_id
Alternate Key	Candidate keys that were not chosen as the Primary Key.	phone_number
Composite Key	A key created by combining 2 or more columns together to achieve uniqueness.	staff_name + staff_title
Foreign Key (FK)	A field in one table that references the Primary Key in another table to build relationships.	staff_id referenced in Payroll table

★ QUICK EXAM CHEAT SHEET

1. RELATIONAL = Tables + Rows/Columns + Enforced Schema 2. SQL = Structured Query Language (DDL, DML, DQL, DCL) 3. NoSQL = Flexible, Schemaless, High Scalability 4. PRIMARY KEY = Uniquely identifies a record in its own table 5. FOREIGN KEY = References a Primary Key in another table to link them 6. CRUD = Create (INSERT), Read (SELECT), Update (UPDATE), Delete (DELETE)